C07 Conventions Équipe à Grande Échelle

Conception Advanced

Synchroniser les pratiques Claude Code dans une équipe de 5+ développeurs

Le problème N×M×P

Dans une équipe de 5 développeurs utilisant 3 outils AI sur 2 OS, vous avez potentiellement 30 configurations différentes. Dans la pratique, après 3 mois, aucun développeur n'a le même CLAUDE.md : Alice a ajouté des règles TypeScript que Bob n'a jamais reçues, Carol a configuré des paths macOS qui cassent sur Linux.

Ce n'est pas un problème de discipline. C'est un problème d'architecture de configuration.

Solution : Profile-Based module assembly

Au lieu d'un fichier monolithique partagé, la structure recommandée décompose les instructions en modules réutilisables assemblés par profil.

```
profiles/          ← 1 fichier YAML par développeur
  alice.yaml
  bob.yaml
modules/           ← Instructions atomiques par topic
  core-standards.md
  typescript-rules.md
  git-workflow.md
  macos-paths.md
skeleton/          ← Template avec placeholders
  claude.md
sync-script.ts     ← Assembleur automatique
```

Un module mis à jour se propage automatiquement à tous les développeurs au prochain `sync`.

Profil développeur (YAML)

```yaml
# profiles/alice.yaml
name: "Alice"
os: "macos"
tools: [claude-code, cursor]
modules:
  core: [core-standards, git-workflow, typescript-rules]
  conditional: [macos-paths, cursor-rules]
```

Alice reçoit uniquement les modules dont elle a besoin. Bob sur Linux sans Cursor reçoit un CLAUDE.md différent, généré depuis les mêmes sources.

Mesurer le résultat

Comparer le volume d'instructions chargé, les échecs de propagation et les résultats acceptés avant et après l'introduction des profils. Aucun benchmark d'équipe n'est fourni ici ; valider cette architecture sur vos configurations.

settings.json versionné = règles d'équipe

Les règles partagées (hooks, permissions autorisées) vont dans `.claude/settings.json` versionné. Les overrides personnels dans `.claude/settings.local.json` gitignore.

```jsonc
// .claude/settings.json (équipe, dans le dépôt)
{
  "hooks": {
    "PreToolUse": [{ "matcher": "Bash", "hooks": [...] }]
  }
}
```

Cette séparation distingue les valeurs partagées des choix personnels. Imposer les restrictions obligatoires par une politique gérée et vérifier la configuration effective.

Seuil de rentabilité

Utiliser un assembleur lorsque la dérive répétée des configurations coûte plus que la maintenance des profils. La taille de l'équipe ne suffit pas à déterminer un seuil de rentabilité.

Pour les équipes plus petites, la stratégie simple reste efficace : un CLAUDE.md commun versionné, un `settings.local.json` par développeur pour les overrides.